

DATABASE SCHEMA DOCUMENT

NSTP Integrated Information System

Group: GROUP FIFTH UNIT

Members: Famero – Capulong – Tinoko – Vallite – De Leon – Baterbonia

Instructor: Prof. Darwin C. Co

Platform: Java SE 21 | Java Swing | MySQL via JDBC

Database: nstp_infosystem (MySQL 8.0 / XAMPP)

Date: May 2026

Includes: ERD · Relational Schema · Table Definitions · Normalization · SQL Views · Data Flow

1. Database Overview

The NSTP Information System uses a locally hosted relational MySQL database named `nstp_infosystem`. The database is structured around a central `STUDENTS` table, from which all other data tables extend through foreign key relationships. This design reflects the core purpose of the system: tracking attendance, engagement, performance, and communications for individual NSTP students enrolled at Pamantasan ng Lungsod ng Maynila.

The database is accessed from the Java application layer exclusively through JDBC (Java Database Connectivity) using the MySQL Connector/J driver. No direct database access occurs from the Presentation Layer. All queries are executed through Data Access Object (DAO) classes in the application.

1.1 Database Specifications

Property	Value
Database Name	<code>nstp_infosystem</code>
Database Engine	InnoDB (with full foreign key enforcement)
Character Set	utf8mb4 (full Unicode + emoji support)
Collation	utf8mb4_general_ci
Server Environment	XAMPP MySQL 8.0+ (local)
JDBC URL	<code>jdbc:mysql://localhost:3306/nstp_infosystem</code>
Total Tables	13 tables (7 core + 6 supporting)
Total Views	5 pre-built views for common queries
Access Method	Java JDBC via DAO classes only
Normalization Level	Third Normal Form (3NF)

2. Entity Relationship Diagram (ERD)

The ERD below shows all core entities, their primary and foreign keys, and the relationships between them. `STUDENTS` is the central entity. All data tables connect to `STUDENTS` through a foreign key (`student_id`), creating one-to-many relationships. The `USERS` table connects to `STUDENTS` through a one-to-one relationship.

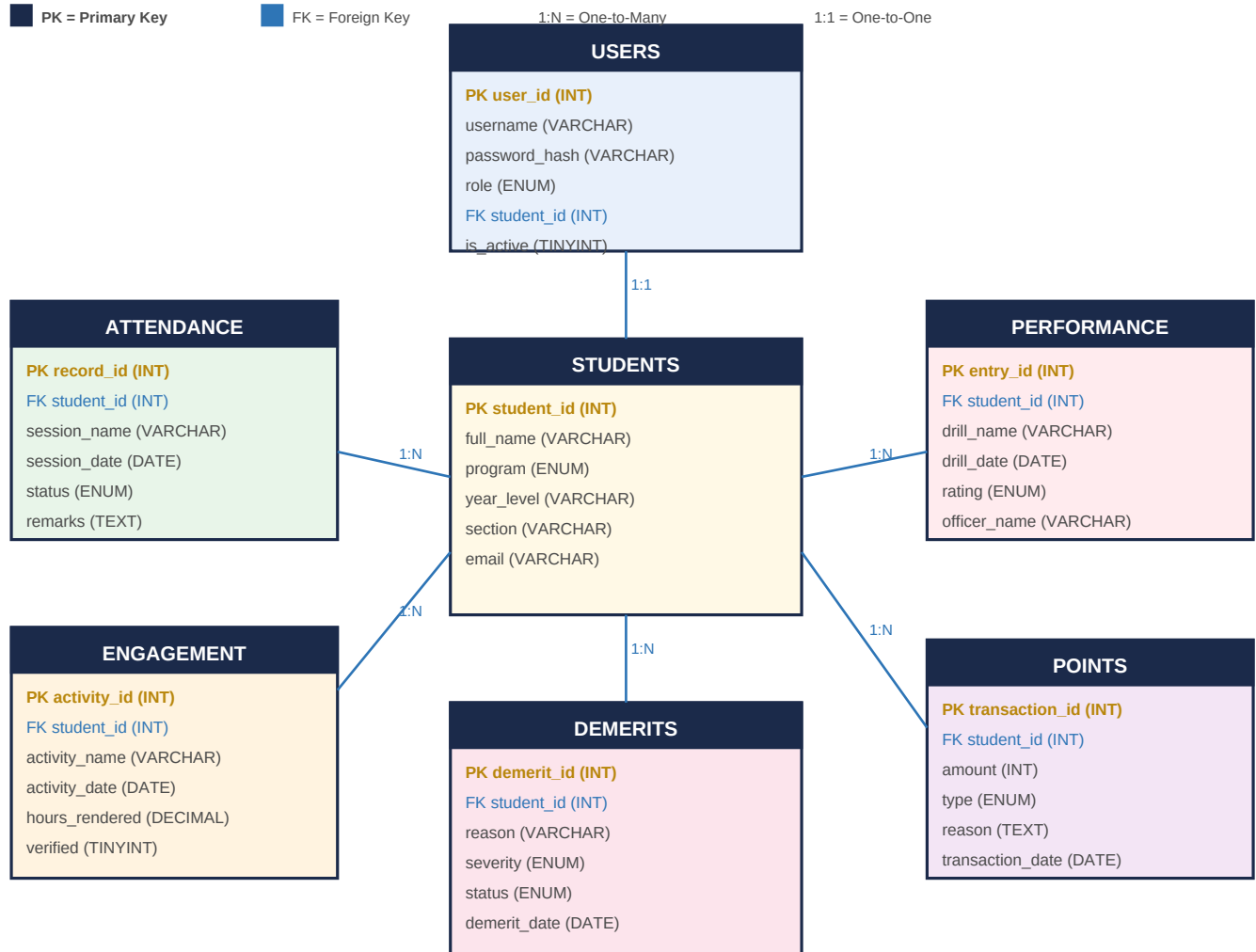


Figure 2.1. Entity Relationship Diagram — NSTP Information System Database (nstp_infosystem)

2.1 Relationship Summary

Relationship	Type	Description
STUDENTS → ATTENDANCE	One-to-Many	One student can have many attendance records across multiple sessions.
STUDENTS → ENGAGEMENT	One-to-Many	One CWTS student can have many community service activity records.
STUDENTS → PERFORMANCE	One-to-Many	One ROTC student can have many drill performance rating entries.
STUDENTS → POINTS	One-to-Many	One student can have many points transactions (awards and deductions).
STUDENTS → DEMERITS	One-to-Many	One ROTC student can have many demerit records assigned by officers.

STUDENTS → NOTIFICATIONS	One-to-Many	One student can receive many system-generated notification alerts.
STUDENTS → DISPUTES	One-to-Many	One student can submit many attendance dispute requests.
USERS ↔ STUDENTS	One-to-One	Each user account is linked to at most one student profile (Admin has no linked student).
ANNOUNCEMENTS ↔ STUDENTS (via read_announcements)	Many-to-Many	Many students can read many announcements. The junction table tracks read status per student per announcement.
ATTENDANCE → DISPUTES	One-to-Many	One attendance record can be referenced by multiple dispute submissions.

3. Database Normalization

The nstp_infosystem database is designed to satisfy Third Normal Form (3NF). Normalization eliminates data redundancy, prevents update anomalies, and ensures data integrity across all tables.

3.1 First Normal Form (1NF)

All tables satisfy 1NF. Each table has a defined primary key. All column values are atomic (no multi-valued or repeating groups in any cell). Every row is uniquely identifiable by its primary key.

- students.program stores a single ENUM value, not a comma-separated list.
- attendance.status is a single atomic ENUM per record.
- Each student-session combination is one row — not combined into a single cell.

3.2 Second Normal Form (2NF)

All tables satisfy 2NF. No table uses a composite primary key where a non-key column depends on only part of the composite key. All non-key columns in every table depend on the entire primary key.

- attendance: record_id is the sole PK. All other columns (session_name, session_date, status) fully describe that specific attendance record.
- points: transaction_id is the sole PK. All columns describe the specific transaction.
- engagement: activity_id is the sole PK. All columns describe one specific activity record.

3.3 Third Normal Form (3NF)

All tables satisfy 3NF. No non-key column is transitively dependent on another non-key column.

- Student profile data (full_name, program, year_level) is isolated in the students table. It is not repeated in attendance, engagement, performance, or any other table.
- User role and credentials are isolated in the users table. Role is not duplicated in session data.
- Activity details (location, category) are stored once per engagement record — not split into a separate lookup table because they are not reused across records in a way that creates redundancy.

■ 3NF is the appropriate normalization level for an academic prototype. Full normalization to BCNF was evaluated but not applied, as ENUM values function as embedded lookup constraints that are sufficient for the scope of this system.

3.4 Referential Integrity Rules

All foreign keys enforce cascading rules to maintain data consistency:

Foreign Key	ON DELETE	ON UPDATE	Effect
attendance.student_id	CASCADE	CASCADE	Deleting a student removes all their attendance records.
engagement.student_id	CASCADE	CASCADE	Deleting a student removes all their engagement records.
performance.student_id	CASCADE	CASCADE	Deleting a student removes all their performance records.

<code>points.student_id</code>	CASCADE	CASCADE	Deleting a student removes all their points transactions.
<code>demerits.student_id</code>	CASCADE	CASCADE	Deleting a student removes all their demerit records.
<code>users.student_id</code>	SET NULL	CASCADE	Deleting a student sets <code>student_id</code> to NULL in the users table (account retained).
<code>*.logged_by / admin_id</code>	SET NULL	CASCADE	Deleting an admin account sets audit fields to NULL (records retained).

4. Table Structure Definitions

4.1 Table: students

Central student profile table. The primary reference for all student-related data. Every data table in the system references this table via student_id.

Column	Data Type	Constraint / Values	Null	Description
student_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique identifier for each student record
full_name	VARCHAR(150)	—	NOT NULL	Student's complete full name
program	ENUM	CWTS / ROTC	NOT NULL	NSTP program enrollment
year_level	VARCHAR(20)	—	NOT NULL	Academic year level (e.g., 2nd Year)
section	VARCHAR(20)	—	NOT NULL	Class section (e.g., BSIT-2A)
contact_no	VARCHAR(20)	—	NULLABLE	Mobile contact number
email	VARCHAR(100)	—	NULLABLE	Institutional email address
created_at	DATETIME	DEFAULT NOW()	NOT NULL	Record creation timestamp

4.2 Table: users

System login accounts. Administrator accounts have student_id = NULL. Student accounts reference their linked profile. One-to-one with students.

Column	Data Type	Constraint / Values	Null	Description
user_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique user account identifier
username	VARCHAR(100)	UNIQUE	NOT NULL	Login username (no duplicates allowed)
password_hash	VARCHAR(255)	SHA-256 hash	NOT NULL	Password stored as SHA-256 hexadecimal hash
role	ENUM	Admin/CWTS_Student/ROTC_Student/ROTC_Senior	NOT NULL	Determines accessible modules
student_id	INT	FK → students.student_id	NULLABLE	NULL for Admin accounts
is_active	TINYINT(1)	1=active, 0=deactivated	NOT NULL	Account access status flag
last_login	DATETIME	—	NULLABLE	Timestamp of most recent login

4.3 Table: attendance

Stores one attendance record per student per session. A composite unique index on (student_id, session_name, session_date) prevents duplicate entries. Used to compute attendance rate per student.

Column	Data Type	Constraint / Values	Null	Description
record_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique attendance record ID
student_id	INT	FK → students.student_id	NOT NULL	The student this record belongs to
session_name	VARCHAR(100)	—	NOT NULL	Name/label of the session (e.g., Week 1)
session_date	DATE	—	NOT NULL	Date the session occurred
status	ENUM	Present/Absent/Late/Excused	NOT NULL	Attendance status assigned by admin
remarks	TEXT	—	NULLABLE	Optional notes from administrator
logged_by	INT	FK → users.user_id	NULLABLE	Admin who logged the record

4.4 Table: engagement

Tracks CWTS community service activities. Verified hours count toward the required minimum (default 20 hours). The CHECK constraint ensures hours_rendered is between 0 and 24 per entry.

Column	Data Type	Constraint / Values	Null	Description
activity_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique activity record ID
student_id	INT	FK → students.student_id	NOT NULL	CWTS student assigned to activity
activity_name	VARCHAR(150)	—	NOT NULL	Name or title of the activity
activity_date	DATE	—	NOT NULL	Date the activity took place
location	VARCHAR(150)	—	NULLABLE	Activity venue or location
category	VARCHAR(80)	—	NULLABLE	Activity type (Environmental, Health, etc.)
hours_rendered	DECIMAL(5,2)	CHECK 0–24	NOT NULL	Number of hours completed
verified	TINYINT(1)	1=verified, 0=unverified	NOT NULL	Admin verification flag

4.5 Table: performance

Stores ROTC drill performance ratings. Rating is an ordered ENUM mapped to integers (Excellent=5, Proficient=4, Satisfactory=3, Needs Improvement=2, Unsatisfactory=1) for average score computation and at-risk detection.

Column	Data Type	Constraint / Values	Null	Description
entry_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique performance entry ID
student_id	INT	FK → students.student_id	NOT NULL	ROTC student being rated
drill_name	VARCHAR(100)	—	NOT NULL	Name of the drill or exercise
drill_date	DATE	—	NOT NULL	Date the drill was conducted
rating	ENUM	Excellent/Proficient/Satisfactory/Needs_Improvement/Unsatisfactory	NOT NULL	Performance rating assigned by officer
remarks	TEXT	—	NULLABLE	Optional notes from rating officer
officer_name	VARCHAR(100)	—	NULLABLE	Name of the rating officer

4.6 Table: points

Transaction ledger for points awarded and deducted. Net balance is computed using SQL: SUM(amount) WHERE type=award minus SUM(amount) WHERE type=deduct. Supports leaderboard and student dashboard display.

Column	Data Type	Constraint / Values	Null	Description
transaction_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique transaction ID
student_id	INT	FK → students.student_id	NOT NULL	Student receiving points change
amount	INT	CHECK > 0	NOT NULL	Points amount (always positive)
type	ENUM	award / deduct	NOT NULL	Transaction direction (add or subtract)
reason	TEXT	—	NOT NULL	Justification for the transaction
transaction_date	DATE	—	NOT NULL	Date the transaction was recorded
admin_id	INT	FK → users.user_id	NULLABLE	Admin who authorized the transaction

4.7 Table: demerits

ROTC demerit records. Severity is weighted (Minor=1, Moderate=2, Major=3) for a weighted demerit score computed in SQL using CASE WHEN expressions. Status tracks appeal and dismissal.

Column	Data Type	Constraint / Values	Null	Description
demerit_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique demerit record ID
student_id	INT	FK → students.student_id	NOT NULL	ROTC student the demerit is assigned to
reason	VARCHAR(200)	—	NOT NULL	Reason for the demerit
severity	ENUM	Minor / Moderate / Major	NOT NULL	Severity level (weights: 1, 2, 3)
notes	TEXT	—	NULLABLE	Additional notes from the officer
officer_name	VARCHAR(100)	—	NULLABLE	Name of the assigning officer
demerit_date	DATE	—	NOT NULL	Date the demerit was recorded
status	ENUM	Active/Appealed/Dismissed	NOT NULL	Current demerit status

4.8 Table: announcements

Admin-posted announcements with audience filtering. Audience filtering ensures CWTS students only see CWTS and All announcements; ROTC students only see ROTC and All. Pinned items appear at top of list.

Column	Data Type	Constraint / Values	Null	Description
announcement_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique announcement ID
title	VARCHAR(200)	—	NOT NULL	Announcement title
body	TEXT	—	NOT NULL	Full announcement text
audience	ENUM	All / CWTS / ROTC	NOT NULL	Target audience for filtering
priority	ENUM	Normal/Important/Urgent	NOT NULL	Priority level for display styling
posted_date	DATETIME	DEFAULT NOW()	NOT NULL	Timestamp of posting
posted_by	INT	FK → users.user_id	NULLABLE	Admin who posted the announcement
is_pinned	TINYINT(1)	1=pinned, 0=normal	NOT NULL	Pinned items sort to top

4.9 Table: notifications

Auto-generated system alerts for individual students. Populated by NotificationService based on conditions: low attendance, new demerits, upcoming events, engagement warnings. The is_read flag tracks unread count for sidebar badge.

Column	Data Type	Constraint / Values	Null	Description
notif_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique notification ID
student_id	INT	FK → students.student_id	NOT NULL	Target student for the notification
type	VARCHAR(80)	—	NOT NULL	Alert type (Performance Alert, Demerit, etc.)
message	TEXT	—	NOT NULL	Notification message text
created_date	DATETIME	DEFAULT NOW()	NOT NULL	When the notification was generated
is_read	TINYINT(1)	0=unread, 1=read	NOT NULL	Read status for badge count

4.10 Table: disputes

Attendance correction requests submitted by students. References both students and a specific attendance record. Admin resolves by updating status and admin_response. Also references the specific attendance record being disputed.

Column	Data Type	Constraint / Values	Null	Description
dispute_id	INT	PK, AUTO_INCREMENT	NOT NULL	Unique dispute ID
student_id	INT	FK → students.student_id	NOT NULL	Student who submitted the dispute
record_id	INT	FK → attendance.record_id	NOT NULL	The attendance record being disputed
reason	TEXT	—	NOT NULL	Explanation from the student
requested_status	ENUM	Present / Excused / Late	NOT NULL	The status the student is requesting
submitted_date	DATE	—	NOT NULL	Date the dispute was filed
status	ENUM	Pending/Approved/Rejected	NOT NULL	Admin resolution status
admin_response	TEXT	—	NULLABLE	Admin reply to the dispute

5. Pre-Built Database Views

Five SQL views are included in the database to simplify common Java DAO queries. These views perform aggregation at the database level, which is more efficient than loading raw data into Java and computing results in application code.

View: vw_points_balance

Computes the net points balance for every student by summing award transactions and subtracting deduction transactions. Used by the student dashboard and leaderboard.

```
SELECT s.student_id, s.full_name, s.program, COALESCE(SUM(CASE WHEN p.type='award' THEN
p.amount ELSE 0 END),0) - COALESCE(SUM(CASE WHEN p.type='deduct' THEN p.amount ELSE 0 END),0) AS
net_balance FROM students s LEFT JOIN points p ON s.student_id=p.student_id GROUP BY
s.student_id;
```

View: vw_attendance_summary

Computes total sessions, present/absent/late/excused counts, and attendance rate percentage for each student. Used by attendance tracker and dashboard.

```
SELECT s.student_id, s.full_name, COUNT(a.record_id) AS total_sessions, SUM(CASE WHEN
a.status='Present' THEN 1 ELSE 0 END) AS present_count, ROUND(SUM(CASE WHEN a.status='Present'
THEN 1 ELSE 0 END) / NULLIF(COUNT(a.record_id),0)*100, 2) AS attendance_rate_pct FROM students s
LEFT JOIN attendance a ON s.student_id=a.student_id GROUP BY s.student_id;
```

View: vw_engagement_totals

Sums total and verified engagement hours per CWTS student. Used by the engagement module to track progress toward the 20-hour minimum.

```
SELECT s.student_id, s.full_name, COALESCE(SUM(e.hours_rendered),0) AS total_hours,
COALESCE(SUM(CASE WHEN e.verified=1 THEN e.hours_rendered ELSE 0 END),0) AS verified_hours FROM
students s LEFT JOIN engagement e ON s.student_id=e.student_id WHERE s.program='CWTS' GROUP BY
s.student_id;
```

View: vw_performance_avg

Computes average performance score per ROTC student using integer mapping (Excellent=5 through Unsatisfactory=1). Used by performance module and at-risk detection logic.

```
SELECT s.student_id, ROUND(AVG(CASE p.rating WHEN 'Excellent' THEN 5 WHEN 'Proficient' THEN 4
WHEN 'Satisfactory' THEN 3 WHEN 'Needs_Improvement' THEN 2 WHEN 'Unsatisfactory' THEN 1 END),2)
AS avg_score FROM students s LEFT JOIN performance p ON s.student_id=p.student_id WHERE
s.program='ROTC' GROUP BY s.student_id;
```


View: vw_demerit_summary

Counts total and active demerits, and computes weighted demerit score (Minor=1, Moderate=2, Major=3) per ROTC student. Used by demerit module and standing badge.

```
SELECT s.student_id, COUNT(d.demerit_id) AS total_demerits, SUM(CASE WHEN d.status='Active'
THEN 1 ELSE 0 END) AS active_demerits, SUM(CASE d.severity WHEN 'Minor' THEN 1 WHEN 'Moderate'
THEN 2 WHEN 'Major' THEN 3 END) AS weighted_score FROM students s LEFT JOIN demerits d ON
s.student_id=d.student_id WHERE s.program='ROTC' GROUP BY s.student_id;
```

6. System Data Flow

The NSTP Information System follows a strict three-layer architecture. Data flows in a defined sequence — from the user through the Presentation Layer, to the Application Layer for processing, and finally to the Data Layer for storage and retrieval. The Presentation Layer never interacts directly with the database.

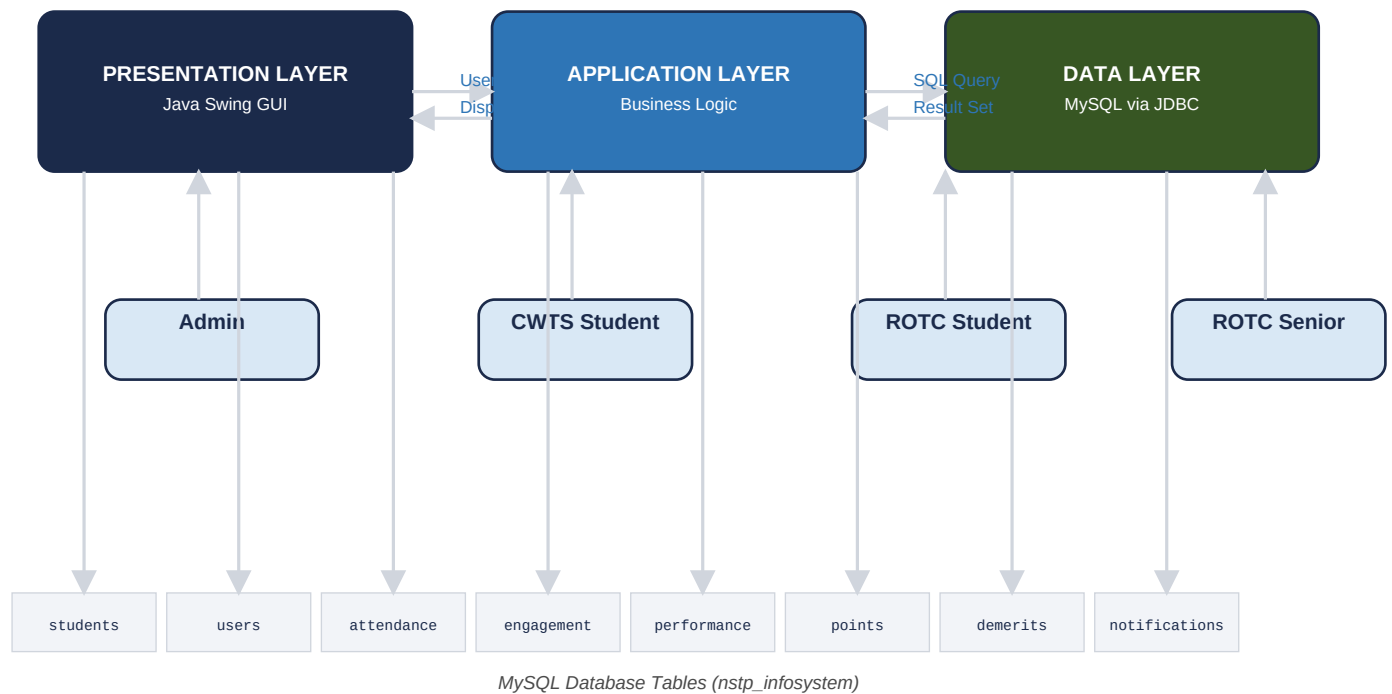


Figure 6.1. Data Flow Diagram — Three-Layer Architecture of the NSTP Information System

6.1 Data Flow Description

Step 1 — User Input: The user (Admin, CWTS Student, ROTC Student, or ROTC Senior) interacts with a Java Swing GUI component — clicking a button, filling a form, or navigating a panel.

Step 2 — Input Validation: The Presentation Layer passes the input to the Application Layer. Service and DAO classes validate the input, check business rules (e.g., negative balance protection), and prepare the SQL query.

Step 3 — Database Operation: The Application Layer sends the SQL query to the MySQL database via JDBC (`DBConnection.getConnection()`). The query is either a SELECT, INSERT, UPDATE, or batch operation.

Step 4 — Result Processing: The database executes the query and returns a ResultSet or row count. The DAO class maps the result into a Java model object (e.g., Student, AttendanceRecord).

Step 5 — Display: The Application Layer passes the processed data back to the Presentation Layer, which renders it in the appropriate Swing component (JTable, JLabel, JProgressBar, etc.).

6.2 Role-Based Access Mapping

Role	Accessible Tables (Read)	Writable Tables
------	--------------------------	-----------------

Administrator	All 13 tables	users, students, attendance, engagement, points, demerits, announcements, calendar_events, feedback
CWTS Student	students (own), attendance (own), engagement (own), points (own), notifications, announcements (All/CWTS), calendar_events	disputes, feedback, read_announcements
ROTC Student	students (own), attendance (own), performance (own), demerits (own), points (own), notifications, announcements (All/ROTC), calendar_events	disputes, feedback, read_announcements
ROTC Senior	students (ROTC), performance (ROTC), demerits (ROTC), attendance (ROTC)	performance, demerits

7. Indexing Strategy

Strategic indexes are applied to columns that are frequently used in WHERE clauses, JOIN conditions, and ORDER BY expressions. Proper indexing ensures that attendance rate computation, leaderboard queries, and notification generation remain performant even as data volume grows.

Table	Index Name	Column(s)	Purpose
attendance	uq_attendance_entry	student_id, session_name, session_date	UNIQUE: prevents duplicate session records per student
attendance	idx_attendance_session	session_name, session_date	Fast lookup of all records for a given session
students	idx_students_program	program	Fast filtering by CWTS or ROTC
points	idx_points_student	student_id	Fast aggregation for balance computation
notifications	idx_notifications_read	is_read	Fast count of unread notifications for sidebar badge
announcements	idx_announcements_pinned	is_pinned, posted_date DESC	Pinned items sort to top of announcement list
calendar_events	idx_calendar_date	event_date	Fast lookup of events by date for calendar rendering

End of Database Schema Document — GROUP FIFTH UNIT — Pamantasan ng Lungsod ng Maynila — May 2026